

Luxury Housing Sales Analytics Dashboard

Project Overview

This project analyzes luxury housing sales data to generate business insights using:

- Python for data cleaning and analysis
- SQLite for database storage
- Streamlit for dashboard visualization
- Plotly for interactive charts

The dashboard provides:

- Market trend analysis
- Builder performance insights
- Booking conversion KPIs
- Configuration demand analysis
- Geographical insights
- Buyer comment insights

Technology Stack

Tool	Purpose
Python	Data Cleaning & Analysis
Pandas	Data Manipulation
SQLite	Database Storage
Streamlit	Dashboard Development
Plotly	Interactive Visualizations
Google Colab	Development Environment

Project Pipeline

Step 1 — Data Collection

The luxury housing dataset was collected in CSV format containing:

- Builder information
 - Booking status
 - Ticket prices
 - Configuration details
 - Micro-market details
 - Buyer comments
 - Sales channel information
-

Step 2 — Data Cleaning

Data preprocessing was performed using Pandas.

Tasks Performed

- Removed null values
- Removed duplicate rows
- Standardized column names
- Converted data types
- Created calculated columns

Example Code

```
import pandas as pd

# Load dataset
_df = pd.read_csv("luxury_housing.csv")

# Remove duplicates
_df.drop_duplicates(inplace=True)

# Handle missing values
_df.dropna(inplace=True)

# Standardize column names
_df.columns = (
    _df.columns
    .str.strip()
```

```
        .str.replace(" ", "_")
    )
```

Step 3 — Feature Engineering

Additional calculated fields were created.

Booking Flag

```
_df['Booking_Flag'] = (
    _df['Booking_Status']
    .apply(lambda x: 1 if x == 'Booked' else 0)
)
```

Price per Sqft

```
_df['Price_per_Sqft'] = (
    _df['Ticket_Price_Cr'] * 10000000
) / _df['Area_Sqft']
```

Step 4 — SQLite Database Creation

The cleaned dataset was stored in SQLite.

Example Code

```
import sqlite3

connection = sqlite3.connect(
    "luxury_housing.db"
)

_df.to_sql(
    "housing_sales",
    connection,
    if_exists="replace",
    index=False
)
```

```
connection.close()
```

SQL Queries Used

Total Revenue

```
SELECT SUM(Ticket_Price_Cr)
FROM housing_sales;
```

Top Builders by Revenue

```
SELECT Builder,
       SUM(Ticket_Price_Cr) AS Revenue
FROM housing_sales
GROUP BY Builder
ORDER BY Revenue DESC;
```

Booking Conversion by Market

```
SELECT Micro_Market,
       AVG(Booking_Flag) * 100 AS Conversion_Rate
FROM housing_sales
GROUP BY Micro_Market;
```

Most Demanded Configuration

```
SELECT Configuration,
       COUNT(*) AS Booking_Count
FROM housing_sales
GROUP BY Configuration
ORDER BY Booking_Count DESC;
```

Streamlit Dashboard Development

The dashboard was developed using Streamlit and Plotly.

Dashboard Features

- Interactive Filters
 - KPI Cards
 - Market Trend Analysis
 - Builder Performance Charts
 - Configuration Demand Analysis
 - Sales Channel Efficiency
 - Buyer Comment Insights
 - Geographical Insights
-

Streamlit Filters (Slicers)

```
selected_builder = st.sidebar.multiselect(
    "Select Builder",
    options=df['Builder'].unique(),
    default=df['Builder'].unique()
)
```

Filters Included

- Builder
 - Quarter
 - Micro Market
 - Configuration
-

KPI Measures

Total Revenue

```
total_revenue = (
    filtered_df['Ticket_Price_Cr'].sum()
)
```

Booking Conversion Rate

```
booking_conversion_rate = (  
    filtered_df['Booking_Flag'].mean() * 100  
)
```

Visualizations Created

Visualization	Purpose
Line Chart	Market Trends
Bar Chart	Builder Performance
Pie Chart	Configuration Demand
Scatter Plot	Amenity Impact
Map Chart	Geographical Insights
KPI Cards	Business Metrics

Business Insights

Market Trends

- Luxury housing bookings increased steadily across major micro-markets.
- Whitefield and Sarjapur showed strong booking growth.

Builder Performance

- Premium builders generated the highest ticket sales.
- Some builders maintained higher average ticket sizes compared to competitors.

Configuration Demand

- 3BHK configurations dominated demand.
- 4BHK properties contributed significantly in premium markets.

Booking Conversion Insights

- Higher amenity scores resulted in better booking conversion rates.
 - Projects with luxury amenities attracted premium buyers.
-

Sales Channel Efficiency

- Online sales channels showed higher conversion efficiency.
 - Digital marketing contributed to improved lead conversion.
-

Buyer Insights

- Buyers preferred ready-to-move luxury apartments.
 - Location and amenities strongly influenced purchase decisions.
-

Streamlit Dashboard Screenshot Section

Add Screenshots Here

Dashboard Home Page

[Insert Screenshot]

KPI Section

[Insert Screenshot]

Market Trend Visualization

[Insert Screenshot]

Builder Performance Chart

[Insert Screenshot]

Configuration Demand Chart

[Insert Screenshot]

Map Visualization

[Insert Screenshot]

Project Architecture

```
graph TD; A[CSV Dataset] --> B[Data Cleaning using Pandas]; B --> C[Feature Engineering]; C --> D[SQLite Database]; D --> E[SQL Querying]; E --> F[Streamlit Dashboard]; F --> G[Business Insights];
```

Challenges Faced

- Handling missing values
 - Standardizing inconsistent column names
 - Streamlit deployment issues in Google Colab
 - Dynamic module loading issues with LocalTunnel
-

Solutions Implemented

- Used Pandas preprocessing techniques
 - Standardized column naming conventions
 - Used Cloudflare Tunnel for stable deployment
 - Added caching for improved dashboard performance
-

Conclusion

This project successfully demonstrated an end-to-end data analytics pipeline for luxury housing sales analysis.

The solution combined:

- Data preprocessing
- SQL database integration
- Interactive dashboarding
- Business intelligence reporting

The dashboard enables stakeholders to:

- Monitor booking trends
 - Analyze builder performance
 - Track conversion rates
 - Understand customer demand patterns
 - Make data-driven business decisions
-

Future Enhancements

- Machine learning prediction models
 - Real-time dashboard updates
 - Advanced NLP analysis on buyer comments
 - Cloud deployment
 - Mobile responsive dashboard
-

GitHub Repository Structure

```
project/
|
├─ data/
├─ app.py
├─ luxury_housing.db
├─ requirements.txt
├─ README.md
├─ screenshots/
└─ notebooks/
```
